

HD44780-compatible. The code needs to be modified accordingly.

The delay or waiting instruction can be implemented easily in C language.

In C programming, the delay is called using the delay() function. For instance, delay(16000) command gives a delay of 16 milliseconds.

**Initialising the LCD.** Before using the LCD, it must be initialised and configured. This is accomplished by sending a number of initialisation instructions to the LCD.

In WINAVR GCC programming given here, the first instruction defines the crystal frequency used in the circuit. This is followed by a standard header file for AVR device-specific I/O definitions and header file for incorporating program space string utilities. The initialisation steps in movm.c file are as follows:

```
#define F_CPU 16000000
#include <avr/io.h>
#include <avr/pgmspace.h>
#include <util/delay.h>
#include "lcd2.h"
#define RATE 250
```

**Clearing the display.** When the LCD is first initialised, the screen should automatically be cleared by the HD44780 controller. This helps to clear the screen of any unwanted text. Clearing the screen also ensures that the text being displayed on the LCD is the one intended for display. An LCD command exists in 'Assembly' to accomplish this function of clearing the screen. Not surprisingly, the AVR GCC function is flexible and easy to implement.

Refer the code given below:

```
lcd_init(LCD_DISP_ON);
lcd_clrscr();
```

Here the first line is for LCD initialisation (turn on the LCD) with cursor 'off.' The second line clears the display routine to clear the LCD screen.

**Writing text to the LCD.** To write text to the LCD, the desired text is put in the memory using the char string[] function as follows:

```
char string[] PROGRAM="WELCOME TO
```

#### ELECTRONICS FOR YOU - NEW DELHI";

The program makes this text scroll on the first line and then the second line of the LCD. The speed of scrolling the text on the LCD screen is defined by "# define RATE 250" in the beginning of the movm.c code. To start with, first the LCD screen is cleared and then the location of the first character to appear on the screen is defined. Getting the text from the program memory and then scrolling it on the LCD screen continuously is achieved using the following code:

```
while(j<=(len-1))
{
    lcd_clrscr();
    lcd_gotoxy(0,0);

    for(k=0,i=1;(k<16) && (pgm_
        read_byte(&string[i])!='\0')
        );k++,i++)
    {
        lcd_putc(pgm_read_
            byte(&string[i]));
    }

    WaitMs(RATE);
    j++;
}
```

#### Compiling and programming.

Compiling the movm.c to generate the movm.hex code is simple. Open AVR Studio4 from the desktop and select new project from 'Project' menu option. Select AVR GCC option and enter the project name. Next, select 'AVR Simulator' and click 'OK' button.

First, copy the three codes (movm.c, lcd2.c and lcd2.h) to your computer. Import the movm.c and lcd2.c files into 'Source Files' option on the left side of the screen. Next, import the lcd2.h file into 'Header Files' option. Select the device as ATmega16 and tick the box against 'Create Hex' option in the project configuration window. Now click 'Rebuild All' option in 'Build' menu. If the code is compiled without error, the movm.hex code is generated automatically under 'Default' folder of the project folder.

To burn the hex code into AT-

#### PARTS LIST

|                                            |                                                      |
|--------------------------------------------|------------------------------------------------------|
| <b>Semiconductors:</b>                     |                                                      |
| IC1                                        | - 7805, 5V regulator                                 |
| IC2                                        | - ATmega16 AVR microcontroller                       |
| LCD1                                       | - 16x2 LCD display module                            |
| BR1A                                       | - 1-amp bridge rectifier module                      |
| LED1                                       | - 5mm light emitting diode                           |
| <b>Resistors (all ¼-watt, ±5% carbon):</b> |                                                      |
| R1                                         | - 470-ohm                                            |
| R2                                         | - 10-kilo-ohm                                        |
| R3                                         | - 47-ohm                                             |
| VR1                                        | - 10-kilo-ohm preset                                 |
| <b>Capacitors:</b>                         |                                                      |
| C1                                         | - 1000µF, 25V electrolytic                           |
| C2                                         | - 0.1µF ceramic                                      |
| C3, C4                                     | - 22pF ceramic                                       |
| C5                                         | - 10µF, 16V electrolytic                             |
| <b>Miscellaneous:</b>                      |                                                      |
| S1                                         | - Tactile switch                                     |
| X <sub>cr</sub>                            | - 16MHz crystal                                      |
| SP16                                       | - 6-pin bergstrip male connector                     |
| X1                                         | - 230V AC primary to 9V, 250mA secondary transformer |

mega16, any standard programmer supporting ATmega16 device can be used. There are four different AVR programming modes:

1. In-system programming (ISP)
2. High-voltage programming
3. Joint test action group (JTAG) programming
4. Program and debug interface programming

Here two options for burning the hex code in standard ISP mode are explained. Some AVR tools that support ISP programming include STK600, STK500, AVRISP mkII, JTAGICC mkII and AVR Dragon. There are also many other AVR programming tools available in the market.

**PonyProg2000 software.** This software along with programmer circuit is available from [www.lancos.com/prog.html](http://www.lancos.com/prog.html) website. After installing PonyProg2000, select the device family as AVR Micro and device type as ATmega16. Now in 'Setup' menu, select 'SI Prog' I/O option for serial programmer and COM port from 'Interface Setup' option. In 'Security and Configuration Bits' option, configure the bits as shown in Fig. 3. Next, open the device file (hex code) and click 'Write All' option to burn the chip.